



Analysis of Regin Malware – Stage 1 Loader

Initial context:

- Regin, a sophisticated APT platform.
- The driver is the first stage, just a loader.
- The location of the next stage is unique and encrypted within the driver's body.
- There are dozens of such samples, need to decrypt them all.

Executive Summary

This report analyzes a suspected Regin Stage 1 kernel-mode loader, a highly stealthy component associated with long-term advanced persistent threat (APT) espionage operations. The sample is a small (11.4 KB), 32-bit native Windows driver compiled in 2007 using Microsoft Visual C++ tooling. Its lack of version metadata, minimal static imports, and deliberate obfuscation indicate a strong emphasis on stealth, stability, and long-term persistence.

The driver's `DriverEntry` routine initializes structured exception handling, disables unloading, and decrypts an embedded configuration stored in the `.data` section using a custom XOR-based routine. This configuration includes a registry path under `HKLM\SYSTEM\CurrentControlSet\Control\RestoreList`, which is decrypted only once using a runtime control flag to limit execution artifacts. Additional data elements include preallocated buffers, benign-looking Windows directory strings, and registry value names used during early-stage environment preparation.

The malware dynamically resolves critical kernel components by locating loaded modules such as `ntoskrnl.exe` and `hal.dll`, and adapts its behavior based on the operating system version using `NtBuildNumber`. Access to kernel internals, including the System Service Descriptor Table (SSDT), enables OS-independent execution without relying on static imports. Overall, the findings confirm this sample functions as a first-stage Regin kernel loader, focused on secure initialization, environment validation, and preparation for subsequent payload stages, reflecting a level of sophistication consistent with highly targeted espionage activity.

Description

The analyzed sample is a small Win32 PE executable (11.41 KB) compiled for 32-bit Windows systems. The file was built using Microsoft Macro Assembler (MASM) and linked with Microsoft Visual C++ 7.1, suggesting deliberate low-level development rather than standard high-level application code. Multiple cryptographic hashes were generated to uniquely identify the sample. The executable exhibits characteristics consistent with a lightweight loader component, including its minimal size and lack of overt functionality. These traits align with the behavior expected from the first-stage loader of the Regin malware platform, which is designed to operate stealthily and facilitate the decryption and in-memory loading of subsequent stages.

File

SHA-256	f1d903251db466d35533c28e3c032b7212aa43c8d64ddf8c5521b43031e69e1e
decrypted file SHA-256:	23115663c967069cb482604700be9f7ed6df4ea1297d453debad1119b0f85b2e

Findings

Attribute	Value
File Name	06665b96e293b23acc80451abb413e50
MD5	06665b96e293b23acc80451abb413e50
SHA-1	9f0dc086875e6b06efe6bb3aadf049ce00f9e486
SHA-256	f1d903251db466d35533c28e3c032b7212aa43c8d64ddf8c5521b43031e69e1e
Vhash	0140466d7e5519z16z17xz
Authentihash	139c80dde62a325f0ae2e2370b801e91caecf57dcf0ac2be7444e99341b9c92
Imphash	a9c1041cccb87f4a7ba3b7048d4e8ad7
Rich PE Header Hash	90c92121c7e7a342a64cd5609234ece1
SSDEEP	192:za3N5H44rW3ias7dUFdELQcsafdPvIw/BZUGJdMr0KqOMwb:zkasBUFuVsafVvIw/BLMcOHb
TLSH	T1C5325D22B6C140F5DA59D8F2AD3A7B39247FAD21633FE5D283340D691D6A841A73708F
File Type	Win32 EXE executable windows pe peexe
Magic	PE32 executable (Intel 80386, MS Windows)
Architecture	x86 (32-bit)
File Size	11.41 KB (11,680 bytes)
TrID Identification	Win32 DLL (34.3%), Win32 EXE (23.4%), Windows Icons Library (10.7%), OS/2 EXE (10.5%), Generic Win/DOS EXE (10.4%)
Detect It Easy	PE32, Compiler: MASM 7.01.4035, Linker: MS Linker 7.10.4035
Magika	PEBIN
PEiD Packer	Microsoft Visual C++ v7.1 EXE

PE Metadata

File Identification and Basic Properties:

Field	Value
SHA-256	F1D903251DB466D35533C28E3C032B7212AA43C8D64DDF8C5521B43031E69E1E
File Name	c:\users\redteam\desktop\regin\06665b96e293b23acc80451abb413e50
File Type	Executable, 32-bit, Native
File Size	11,680 bytes
Entropy	6.524
File Version	N/A
File Description	N/A
Digital Signature	Microsoft Linker 7.10 Visual Studio Microsoft Visual C++ 6.0-8.0

PE Headers

Field	Value
DOS Header (Hex)	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00 B8 00 00 00 00 00 00 40 00 00 00 00 00 00
DOS Header (Text)	MZ.....@.....
Architecture	Intel x86 (32-bit)
Subsystem	Native

Entry Point Information

Field	Value
Entry Point Offset	0x00000814

Field	Value
Entry Point Section	.text
Entry Point Bytes (Hex)	6A 0C 68 10 03 01 00 E8 74 00 00 00 83 65 FC 00 8B 45 08 C7 40 34 06 08 01 00 80 3D 68 25 01 00

Sections

Section Name	Virtual Address	Virtual Size (bytes)	Raw Size (bytes)	Entropy	MD5	Chi-Sq (Chi²)
.text	0x00000280 (640)	8,904	8,928	6.52	e9285367382d63f5e65445b78a688b5e	49,665
.data	0x00002560 (9,568)	824	832	7.33	57db72556a0e508de756687d48351a93	1,608.6
INIT	0x000028A0 (10,400)	586	608	4.86	288e1b4ff839ac154cb20074ca668049	13,850
.reloc	0x00002B00 (11,008)	644	672	3.16	8a24633536aa0a73063171a85bac00b4	68,508

Compilation and Time Stamp Information

Field	Value
Compiler Timestamp	Thu Dec 06 01:23:00 2007 (UTC)
Debug Timestamp	N/A
Resource Timestamp	N/A
Import Timestamp	N/A
Export Timestamp	N/A

Description:

When we look at the parts of the program we see that it is put together in an organized way. The part that has the code is pretty mixed up which is what we usually see in code that someone wrote by hand or tried to make very small. This kind of code is often used in the parts of the program that load things. The part that stores data is more mixed up which makes us think that it might have secret or packed up information like settings or another hidden program. There is also a part that starts things up which is not typical and it means that the bad program has its own way of getting started. The part that helps the program move to places on the computer is pretty simple, which is what we expect to see. The program and the bad program are the thing in this case the malware. The malware has its lifecycle. The parts of the malware, like the .text section and the .data section and the INIT section and the .reloc section are all important to understand how the malware works. Overall, the section layout and entropy distribution support the assessment that this sample functions as a minimal first-stage loader designed for stealth and controlled execution rather than direct malicious activity.

Static Analysis

Imports /Resolved / Referenced Kernel APIs

Function Name	Flag	Resolution Type	Ordinal	Address (RVA)	Thunk Address	Module
wcslen	-	implicit	-	0x00002A2C	0x00002A2C	ntoskrnl.exe
wscpy	-	implicit	-	0x000029F8	0x000029F8	ntoskrnl.exe
strchr	-	implicit	-	0x00002A4C	0x00002A4C	ntoskrnl.exe
strncpy	-	implicit	-	0x00002A84	0x00002A84	ntoskrnl.exe
strncmp	-	implicit	-	0x00002A72	0x00002A72	ntoskrnl.exe
strchr	-	implicit	-	0x00002A42	0x00002A42	ntoskrnl.exe
atoi	-	implicit	-	0x00002A7C	0x00002A7C	ntoskrnl.exe

Function Name	Flag	Resolution Type	Ordinal	Address (RVA)	Thunk Address	Module
_strnicmp	-	implicit	-	0x00002A36	0x00002A36	ntoskrnl.exe
_snprintf	-	implicit	-	0x00002A9E	0x00002A9E	ntoskrnl.exe
ZwQueryValueKey	x	implicit	-	0x00002966	0x00002966	ntoskrnl.exe
ZwQuerySystemInformation	x	implicit	-	0x00002A56	0x00002A56	ntoskrnl.exe
ZwQueryInformationFile	x	implicit	-	0x00002A12	0x00002A12	ntoskrnl.exe
ZwOpenKey	-	implicit	-	0x00002978	0x00002978	ntoskrnl.exe
ZwCreateFile	-	implicit	-	0x00002A02	0x00002A02	ntoskrnl.exe
ZwClose	-	implicit	-	0x0000295C	0x0000295C	ntoskrnl.exe
RtlUnwind	-	implicit	-	0x000029C6	0x000029C6	ntoskrnl.exe
RtlInitAnsiString	-	implicit	-	0x000029A4	0x000029A4	ntoskrnl.exe
RtlFreeUnicodeString	-	implicit	-	0x00002944	0x00002944	ntoskrnl.exe
RtlAnsiStringToUnicodeString	-	implicit	-	0x00002984	0x00002984	ntoskrnl.exe
NtBuildNumber	-	implicit	-	0x00002A8E	0x00002A8E	ntoskrnl.exe
KeServiceDescriptorTable	x	implicit	-	0x00002AAA	0x00002AAA	ntoskrnl.exe
KeQueryPerformanceCounter	-	implicit	-	0x00002AC6	0x00002AC6	HAL.dll
ExFreePool	-	implicit	-	0x000029D2	0x000029D2	ntoskrnl.exe
ExAllocatePoolWithTag	-	implicit	-	0x000029E0	0x000029E0	ntoskrnl.exe

Hex editor

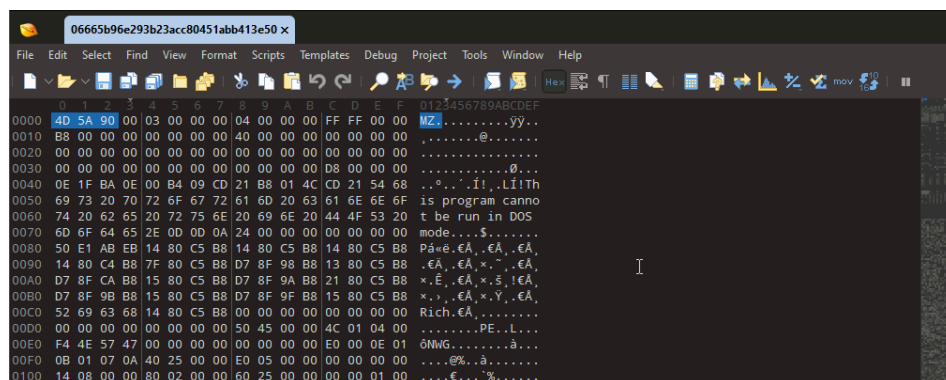


Fig 1 - MZ header in the beginning

IDA Analysis

IDA recognized the entry point as DriverEntry.

```
; Attributes: bp-based frame

; NTSTATUS __stdcall DriverEntry(PDRIVER_OBJECT DriverObject, PUNICODE_STRING RegistryPath)
public DriverEntry
DriverEntry proc near

var_1C= dword ptr -1Ch
ms_exc= CPPEH_RECORD ptr -18h
DriverObject= dword ptr 8
RegistryPath= dword ptr 0Ch

; __unwind { // __SEH_prolog
push    0Ch
push    offset stru_10310
call    __SEH_prolog
```

Fig 2 - Entry point

DriverEntry function (mcd.h) - The **DriverEntry** miniport driver routine is called when the miniport driver is loaded.

Syntax

```
C++  
NTSTATUS DriverEntry(  
    [in] PDRIVER_OBJECT DriverObject,  
    [in] PUNICODE_STRING RegistryPath  
);
```

[Copy](#)

Parameters

[in] DriverObject

A pointer to an opaque structure to be used as the first parameter when this routine calls the [AtaPortInitializeEx](#) routine.

[in] RegistryPath

A pointer to an opaque structure to be used as the second parameter when this routine calls the [AtaPortInitializeEx](#) routine.

Return value

The return values for this routine are recommended to mirror the return values of [AtaPortInitializeEx](#).

Fig 3 - msdn DriverEntry

- One is a pointer to structure of type PDRIVER_OBJECT
- Pointer to structure of type PUNICODE_STRING and it's a registry path. that is unique for this driver.

Beginning of the code goes immediately goes into loop. XORing some bytes that IDA renamed as SourceString.

```

.text:00010839
.text:00010839 loc_10839: ; CODE XREF: DriverEntry+40↓j
.text:00010839 mov     [ebp+var_1C], esi
.text:0001083C cmp     esi, 2EEh
.text:00010842 jnb     short loc_10863
.text:00010844 lea     eax, [SourceString[esi]] ; "oCR"
.text:0001084A mov     ecx, esi
.text:0001084C and     ecx, 7
.text:0001084F movsx   ecx, ds:byte_1080A[ecx]
.text:00010856 xor     edx, edx
.text:00010858 mov     dl, [eax]
.text:0001085A xor     ecx, edx
.text:0001085C xor     ecx, esi
.text:0001085E mov     [eax], cl

```

Fig 4 - xor SourceString

Following `SourceString` Encrypted data followed by more encrypted data.

```

.data:00012569 ; char SourceString[3]
.data:00012569 SourceString db 'oCR',0AEh,'kkj',02h,62h,45h,52h,80h,'Ix_',0C6h,46h,']T',80h,'A|K'
.data:00012569 ; DATA XREF: sub_106C0+CA↑o
.data:00012569 ; DriverEntry+30↑o
.data:00012580 db 00Dh,77h,'Jz',83h,'HeH',0CCh,50h,'AY',080h,'pWn',003h,7Eh,'Mc',82h
.data:00012595 db 'e^b',0FAh,'lM{',88h,'w[z',0FFh,'yLc',088h,'lT',6,98h,'sQh',0A9h,'b'
.data:000125AE db 'X~',0E0h,78h,59h,'_',0A1h,'jPv',0E8h,63h,41h,'G',009h,'rHn',0F0h,68h,49h,4Fh
.data:000125C4 db 081h,'z@f',0F8h,'sqw',89h,'Bx^',0C0h,58h,'y',7Fh,81h,'jPv',0C8h,43h
.data:000125DA db 'ag',99h,'RhN',0D0h,48h,'io',91h,'Z' F91,'u',97h,'i',0A2h,98h,0Eh,' '
.data:000125F1 db 088h,99h,9Fh,'a',0AAh,90h,0B6h,'(',0A3h,81h,87h,'y',0B2h,88h,0AEh,'0'
.data:00012601 db 0A8h,89h,8Fh,'q',0BAh,80h,0A6h,'8',93h,0B1h,0B7h,'I',82h,0B8h,9Eh,0
.data:00012611 db 9Bh,009h,0BFh,'A',8Ah,0B0h,96h,8,83h,0A1h,0A7h,'Y',92h,0A8h,8Eh,10h
.data:00012621 db 8Bh,0A9h,0AFh,'Q',9Ah,0A0h,86h,18h,0F3h,0D1h,0D7h,29h,0E2h,0D8h,0FEh
.data:00012630 db ' '
.data:00012631 ; char byte_12631
.data:00012631 byte_12631 db 0ADh ; DATA XREF: sub_106C0:loc_10785↑o
.data:00012632 db 0B8h
.data:00012633 db 0B8h
.data:00012634 db 44h ; D
.data:00012635 db 85h
.data:00012636 db 92h

```

Fig 5 - Encrypted data conversion to Array

Going into `sub_106C0` next call is using another buffer.

`.text:000106E0 push offset unk_12663` at this address

`.data:00012663 unk_12663 db 0D3h ; DATA XREF: sub_106C0+20↑o`

```

.text:000106C0 P = dword ptr -0Ch
.text:000106C0 var_8 = dword ptr -8
.text:000106C0 var_4 = dword ptr -4
.text:000106C0
.text:000106C0 push     ebp
.text:000106C1 mov     ebp, esp
.text:000106C3 sub     esp, 2Ch
.text:000106C6 push     ebx
.text:000106C7 xor     eax, eax
.text:000106C9 push     esi
.text:000106CA push     edi
.text:000106CB mov     [ebp+P], eax
.text:000106CE mov     [ebp+var_20], eax
.text:000106D1 mov     [ebp+var_10], eax
.text:000106D4 mov     [ebp+var_24], eax
.text:000106D7 mov     [ebp+var_18], eax
.text:000106DA mov     [ebp+var_14], eax
.text:000106DD mov     [ebp+var_28], eax
.text:000106E0 push     offset unk_12663
.text:000106E5 lea     eax, [ebp+var_8]
.text:000106E8 push     eax
.text:000106E9 lea     ebx, [ebp+var_4]
.text:000106EC lea     esi, [ebp+var_20]
.text:000106EF lea     edi, [ebp+P]
.text:000106F2 call    sub_10400
.text:000106F7 test     al, al
.text:000106F9 jz      loc_10785

```

Fig 6 - address 0x106E0 calling buffer.

Almost every important information in the code is using one or another buffer that looks like encrypted strings.

Decryption

decryption program template:

```
#!/usr/bin/python3

from decoder_core import *
from Crypto.Cipher import *

class Decoder(TrainingDecoder):
    def __init__(self):
        # MD5 hash of the correctly decrypted file
        super().__init__()

    def decode(self):
        # self.data is a bytearray with the contents of the file

        # 1. Set the proper file offset of the encrypted block
        encrypted_block_offset = REPLACE_WITH_THE_SOLUTION
        # 2. Set the proper length of the encrypted block
        encrypted_block_len = REPLACE_WITH_THE_SOLUTION
        # 3. Set the proper file offset of the decryption key
        key_offset = REPLACE_WITH_THE_SOLUTION
        # 4. Set the proper length of the decryption key
        key_len = REPLACE_WITH_THE_SOLUTION

        key = self.data[key_offset:key_offset+key_len]
        for i in range(0,encrypted_block_len):
            self.data[encrypted_block_offset+i] = ( self.data[encrypted_block_offset+i] ^ key[i%key_len] ^ i ) & 0xFF

        # Check if the results are correct
        self.check_results()

# Create the decryptor object. Automation happens in __init__()
Decoder()
```

```
.text:00010814 ; __unwind { // __SEH_prolog
.text:00010814 push 0Ch
.text:00010816 push offset stru_10310
.text:00010818 call __SEH_prolog
.text:00010820 ; __try { // __except at loc_1087F
.text:00010820 and [ebp+ms_exc.registration.TryLevel], 0
.text:00010824 mov eax, [ebp+DriverObject]
.text:00010827 mov dword ptr [eax+34h], offset nullsub_1
.text:0001082E cmp byte_12568, 0
.text:00010835 jz short loc_10863
.text:00010837 xor esi, esi
.text:00010839 loc_10839: ; CODE XREF: DriverEntry+4D1j
.text:00010839 mov [ebp+var_1C], esi
.text:0001083C cmp esi, 2EEh
.text:00010842 jnb short loc_10863
.text:00010844 lea eax, SourceString[esi] ; "oCRkKjIxA[KzHeH^y^pwn"...
.text:0001084A mov ecx, esi
.text:0001084C and ecx, 7
.text:0001084F movsx ecx, ds:byte_1080A[ecx]
.text:00010856 xor edx, edx
.text:00010858 mov dl, [eax]
.text:0001085A xor ecx, edx
.text:0001085C xor ecx, esi
.text:0001085E mov [eax], cl
.text:00010860 inc esi
.text:00010861 jmp short loc_10839
```

Fig 7 - Counter loop

The register **ESI** is used as the loop counter, initialized to zero and incremented on each iteration until it reaches the upper bound of **0x2EE**, defining the size of the decrypted buffer.

encrypted_block_len = 0x2EE

Convert Virtual address to the file offset

IDA

```

.data:00012569 byte_12569      dd 1                ; DATA XREF: DriverEntry+12569
.data:00012569 ; char SourceString[3]
.data:00012569 SourceString db 'eCR',0AEh,'kkj',0F2h,62h,45h,52h,80h,'Ix',0C6h,46h,'JT',80h,'A|K'
.data:00012569 ; DATA XREF: sub_106C0+CAto
.data:00012569 ; DriverEntry+30To
.data:00012580 db 00Dh,77h,'Jz',83h,'HeH',0CCh,50h,'AY',08Dh,'pwr',003h,7Eh,'Mc',82h
.data:00012585 db 'e'b',0FAh,'lM',8Bh,'w[z',0FFh,'yLc',088h,'IT',6,98h,'sQW',0A9h,'b'
.data:0001258E db 'X-',0E0h,7Bh,59h,'',0A1h,'jPv',0E8h,63h,41h,'G',089h,'rHn',0F0h,6Bh,49h,4Fh
.data:00012594 db 0B1h,'z@f',0F8h,'Sqw',89h,'Bx^',0C0h,5Bh,'y',7Fh,81h,'3pV',0C8h,43h
.data:0001259A db 'ag',99h,'RHN',0D0h,4Bh,'io',91h,'Z' F91,'u',97h,'i',0A2h,98h,0Eh,' '
.data:000125A0 db 08Bh,99h,9Fh,'a',0AAh,90h,086h,'(',0A3h,81h,87h,'y',0B2h,88h,0AEh,'0'
.data:000125A8 db 0ABh,89h,8Fh,'q',0BAh,80h,0A6h,'8',93h,0B1h,0B7h,'I',82h,0B8h,0Eh,0
.data:000125B0 db 9Bh,0B9h,0BFh,'A',8Ah,0B0h,96h,8,83h,0A1h,0A7h,'Y',92h,0A8h,8Eh,10h
.data:000125B8 db 8Bh,0A9h,0AFh,'Q',9Ah,0A0h,86h,18h,0F3h,0D1h,0D7h,29h,0E2h,0D8h,0FEh
.data:000125C0 db ''
.data:00012631 ; char byte_12631
.data:00012631 byte_12631 db 0ADh                ; DATA XREF: sub_106C0:loc_10785fo
.data:00012632 db 080h
.data:00012633 db 08Bh
.data:00012634 db 44h ; 0
.data:00012635 db 85h
.data:00012636 db 92h
.data:00012637 db 97h
.data:00012638 db 18h
.data:00012639 db 86h
.data:0001263A db 0C1h
.data:0001263B db 0C7h
.data:0001263C db 39h ; 9
.data:0001263D db 0F2h
.data:0001263E db 0C8h

```

Fig 8 - encrypted block offset

encrypted_block_offset = 0x2569

The **XOR key** in this routine is a **small rolling key** stored at **byte_1080A**, combined with the **loop counter** (**ESI**).

```

.text:00010804
.text:00010804 ; -----
.text:00010805 align 2
.text:00010806 ; [00000003 BYTES: COLLAPSED FUNCTION nullsub_1]
.text:00010809 db 0CCh
.text:0001080A unk_1080A db 33h ; 3 ; DATA XREF: DriverEntry+3B4r
.text:0001080B db 10h
.text:0001080C db 15h
.text:0001080D db 0EAh
.text:0001080E db 26h ; 8
.text:0001080F db 1Dh
.text:00010810 db 38h ; 8
.text:00010811 db 0A7h
.text:00010812 db 0
.text:00010813 db 0CCh
.text:00010814

```

Fig 9 - byte_1080A


```

.text:00010805 align 2
.text:00010806 ; [00000003 BYTES: COLLAPSED FUNCTION nullsub_1]
.text:00010809 db 0CCh
.text:0001080A unk_1080A db 33h ; 3 ; DATA XREF: DriverEntry+384r
.text:0001080B db 10h
.text:0001080C db 15h
.text:0001080D db 0EAh
.text:0001080E db 26h ; 8
.text:0001080F db 1Dh
.text:00010810 db 38h ; 8
.text:00010811 db 0A7h
.text:00010812 db 0
.text:00010813 db 0CCh
.text:00010814 ; ===== SUBROUTINE =====
.text:00010814 ; Attributes: bp-based frame
.text:00010814 ; NTSTATUS __stdcall DriverEntry(PDRIVER_OBJECT DriverObject, PUNICODE_STRING RegistryPath)
.text:00010814 public DriverEntry
.text:00010814 proc near ; DATA XREF: HEADER:00010100+0
.text:00010814
.text:00010814 var_1C = dword ptr -1Ch
.text:00010814 ms_exc = CPPEH_RECORD ptr -18h
.text:00010814 DriverObject = dword ptr 8
.text:00010814 RegistryPath = dword ptr 0Ch
.text:00010814
.text:00010814 ; __unwind { // __SEH_prolog
.text:00010814 push 0Ch
.text:00010816 push offset stru_10310
.text:0001081B call __SEH_prolog
0000080A 0001080A: .text:unk_1080A (Synchronized with Hex View-1)

```

Fig 10 - key offset

key_offset = 0x80A

```

.text:00010839 mov [ebp+var_1C], esi
.text:0001083C cmp esi, 2EEh
.text:00010842 jnb short loc_10863
.text:00010844 lea eax, SourceString[esi] ; "oCRkKjIX]T
.text:0001084A mov ecx, esi
.text:0001084C and ecx, 111b
.text:0001084F movsx ecx, byte ptr ds:unk_1080A[ecx]
.text:00010856 xor edx, edx
.text:00010858 mov dl, [eax]
.text:0001085A xor ecx, edx

```

Fig 11 - Converting 7 to binary is 111b

We're going to have values starting from 0 and ending in 7, including 7.

So the length of the key is 8.

```

[kant@APPLEs-MacBook-Pro ~/D/regin> ls (cyber)
06665b96e293b23acc80451abb413e50* regin_driver.py*
decoder_core/
[kant@APPLEs-MacBook-Pro ~/D/regin> python3 regin_driver.py 06665b96e293b23acc80451abb413e50
The MD5 checksum of the decoded file is:
f8d94c86b15b62d5d76b9846fad5ae59
Written the results to 06665b96e293b23acc80451abb413e50.dec
[kant@APPLEs-MacBook-Pro ~/D/regin> (cyber)

```

Fig 12 - Successfully decrypted

File Differences

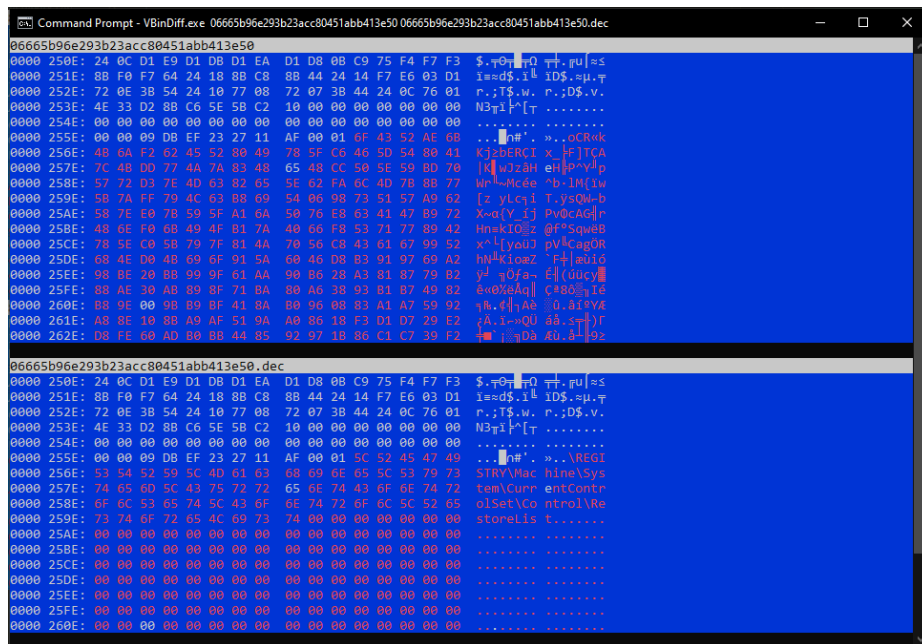


Fig 13 - First difference

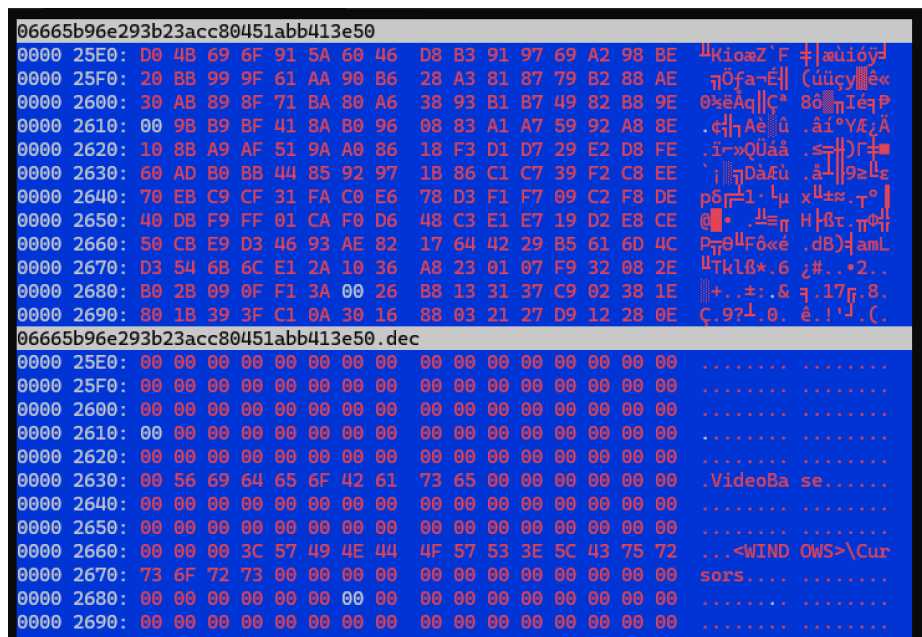


Fig 14 - Second difference

Decrypted File Analysis

Basic Properties

Attribute	Value
File name	06665b96e293b23acc80451abb413e50.dec
MD5	f8d94c86b15b62d5d76b9846fad5ae59
SHA-1	41a24d8697c06890cd0cf96ecb4ee30904ef5087
SHA-256	23115663c967069cb482604700be9f7ed6df4ea1297d453debad1119b0f85b2e

Attribute	Value
Vhash	0140466d1e5519z16z17xz
Authentihash	8ccd332ab1c6585caafbb58fb31f03be285c8342ce32ff4a8df0120e50c72163
Imphash	a9c1041cccb87f4a7ba3b7048d4e8ad7
Rich PE Header Hash	90c92121c7e7a342a64cd5609234ece1
SSDEEP	192:za3N5H44rW3ias7dUFdELQcsafdPvIw/BZUGJdMr0KqOMGb:zkasBUFuVsafVvIw/BLMc09b
TLSH	T158325C52B6C040F5DB59D8F29D277B3924BE9E21632BE6D387340D680D9A942A73708F
File Type	Win32 EXE
Magic	PE32 executable (Intel 80386, MS Windows)
Architecture	x86 (32-bit)
File Size	11.41 KB (11,680 bytes)
TrID Identification	Win32 DLL (34.3%), Win32 EXE (23.4%), Windows Icons Library (10.7%), OS/2 EXE (10.5%), Generic Win/DOS EXE (10.4%)
Detect It Easy	PE32, Compiler: MASM 7.01.4035, Linker: Microsoft Linker 7.10.4035
Magika	PEBIN
PEiD Packer	Microsoft Visual C++ v7.1 EXE

Sections

Section Name	Virtual Address	Virtual Size (bytes)	Raw Size (bytes)	Entropy	MD5	Chi-Sq (Chi²)
.text	0x00000280 (640)	8,904	8,928	6.52	e9285367382d63f5e65445b78a688b5e	49,665
.data	0x00002560 (9,568)	824	832	1.36	22b3dd607e91c85ddef807a0441042f4	154,011
INIT	0x000028A0 (10,400)	586	608	4.86	288e1b4ff839ac154cb20074ca668049	13,850
.reloc	0x00002B00 (11,008)	644	672	3.16	8a24633536aa0a73063171a85bac00b4	68,508

MITRE ATT&CK Tactics and Techniques

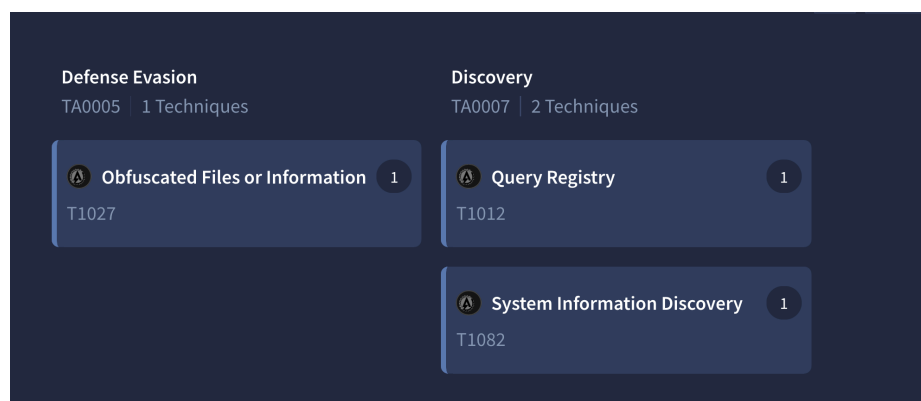


Fig 15 - MITRE

Sample Comparison – File Metadata and Hashes

Attribute	06665b96e293b23acc80451abb413e50	06665b96e293b23acc80451ab
MD5	06665b96e293b23acc80451abb413e50	f8d94c86b15b62d5d76b9846fad5ae
SHA-1	9f0dc086875e6b06efe6bb3aadf049ce00f9e486	41a24d8697c06890cd0cf96ecb4ee3
SHA-256	f1d903251db466d35533c28e3c032b7212aa43c8d64ddf8c5521b43031e69e1e	23115663c967069cb482604700be9f

Attribute	06665b96e293b23acc80451abb413e50	06665b96e293b23acc80451ab
Vhash	0140466d7e5519z16z17xz	0140466d1e5519z16z17xz
Authentihash	139c80dde62a325f0ae2e2370b801e91caecf57dcf0ac2be7444e99341b9c92	8ccd332ab1c6585caafbb58fb31f03b
Imphash	a9c1041cccb87f4a7ba3b7048d4e8ad7	a9c1041cccb87f4a7ba3b7048d4e8a
Rich PE Header Hash	90c92121c7e7a342a64cd5609234ece1	90c92121c7e7a342a64cd5609234ec
SSDEEP	192:za3N5H44rW3ias7dUFdELQcsafdPvlw/BZUGJdMrOKqOMwb:zkasBUFuVsafVvlw/BLMcOHb	192:za3N5H44rW3ias7dUFdELQcsafi
TLSH	T1C5325D22B6C140F5DA59D8F2AD3A7B39247FAD21633FE5D283340D691D6A841A73708F	T158325C52B6C040F5DB59D8F29C
File Type	Win32 EXE	Win32 EXE
Magic	PE32 executable (Intel 80386, MS Windows)	PE32 executable (Intel 80386, M
Architecture	x86 (32-bit)	x86 (32-bit)
File Size	11.41 KB (11,680 bytes)	11.41 KB (11,680 bytes)
TrID Identification	Win32 DLL (34.3%), Win32 EXE (23.4%), Windows Icons Library (10.7%), OS/2 EXE (10.5%), Generic Win/DOS EXE (10.4%)	Same
Compiler / Linker	MASM 7.01.4035 / MS Linker 7.10.4035	MASM 7.01.4035 / MS Linker 7.1
Magika	PEBIN	PEBIN
PEiD Packer	Microsoft Visual C++ v7.1 EXE	Microsoft Visual C++ v7.1 EXE

Imported Kernel Functions

Module	Imported Function
ntoskrnl.exe	_snprintf
ntoskrnl.exe	_strnicmp
ntoskrnl.exe	atoi
ntoskrnl.exe	ExAllocatePoolWithTag
ntoskrnl.exe	ExFreePool
ntoskrnl.exe	KeServiceDescriptorTable
ntoskrnl.exe	NtBuildNumber
ntoskrnl.exe	RtlAnsiStringToUnicodeString
ntoskrnl.exe	RtlFreeUnicodeString
ntoskrnl.exe	RtlInitAnsiString
HAL.dll	KeQueryPerformanceCounter

IDA Analysis of Decrypted file

```

.text:00010014 ; Attributes: bp-based frame
.text:00010014 ; NTSTATUS __stdcall DriverEntry(PDRIVER_OBJECT DriverObject, PUNICODE_STRING RegistryPath)
.text:00010014 public DriverEntry
.text:00010014 DriverEntry proc near ; DATA XREF: HEADER:00010100f0
.text:00010014 var_1C = dword ptr -1Ch
.text:00010014 ms_exc = CPPEH_RECORD ptr -10h
.text:00010014 DriverObject = dword ptr 8
.text:00010014 RegistryPath = dword ptr 0Ch
.text:00010014
.text:00010014 ; __unwind { // __SEH_prolog
.text:00010014 push 0Ch
.text:00010014 push offset stru_10310
.text:00010016 call __SEH_prolog
.text:00010018 ; __try { // __except at loc_10037
.text:00010020 and [ebp+ms_exc.registration.TryLevel], 0
.text:00010024 mov eax, [ebp+DriverObject]
.text:00010027 mov dword ptr [eax+34h], offset nullsub_1
.text:0001002E cmp byte_12568, 0
.text:00010035 jz short loc_10063
.text:00010037 xor esi, esi
.text:00010039
.text:00010039 loc_10039: mov [ebp+var_1C], esi ; CODE XREF: DriverEntry+40d3
.text:00010039 cmp esi, 2EEh
.text:0001003C jnb short loc_10063
.text:00010042 lea eax, SourceString[esi] ; "\\REGISTRY\\Machine\\System\\CurrentControlSet\\Control\\RestoreList",0
.text:00010044 mov ecx, esi

```

Fig 16 - Loading decrypted binary in IDA

Same Function **DriverEntry** , First block of data that was encrypted is now a readable registry location

```

.data:00012566 db 0AFh
.data:00012567 db 0
.data:00012568 byte_12568 db 1 ; DATA XREF: DriverEntry+1Afr
.data:00012569 ; char SourceString[]
.data:00012569 SourceString db '\\REGISTRY\\Machine\\System\\CurrentControlSet\\Control\\RestoreList',0
.data:00012569 SourceString db '\\REGISTRY\\Machine\\System\\CurrentControlSet\\Control\\RestoreList',0 ; DATA XREF: sub_106C0+CAt0
.data:00012569 SourceString db '\\REGISTRY\\Machine\\System\\CurrentControlSet\\Control\\RestoreList',0 ; DriverEntry+30f0
.data:00012568 db 89h dup(0)
.data:00012569 ; char aVideoBase[]
.data:00012569 aVideoBase db 'VideoBase',0 ; DATA XREF: sub_106C0:loc_10785f0
.data:00012568 db 28h dup(0)
.data:00012569 aWindowsCursors db '<WINDOWS>\\Cursors',0 ; DATA XREF: sub_106C0+20f0
.data:00012568 db 0E8h dup(0)
.data:00012569 aWindowsFonts db '<WINDOWS>\\Fonts',0 ; DATA XREF: sub_106C0+51f0
.data:00012568 db 0E8h dup(0)
.data:00012569 off_12858 dd offset aNtoskrnlExe_2 ; DATA XREF: sub_11042+A1fr
.data:00012568 ; "ntoskrnl.exe"
.data:00012569 aHalDll_1 dd offset aHalDll_1 ; "hal.dll"
.data:00012568 ; "disk.sys"
.data:00012569 aDiskSys dd offset aDiskSys
.data:00012568 ; "atapi.sys"
.data:00012569 aAtapiSys dd offset aAtapiSys

```

Fig 17 - Registry location

We have two folders name supposedly two folder's name

```

.data:00012566 db 0AFh
.data:00012567 db 0
.data:00012568 byte_12568 db 1 ; DATA XREF: DriverEntry+1Afr
.data:00012569 ; char SourceString[]
.data:00012569 SourceString db '\\REGISTRY\\Machine\\System\\CurrentControlSet\\Control\\RestoreList',0
.data:00012569 SourceString db '\\REGISTRY\\Machine\\System\\CurrentControlSet\\Control\\RestoreList',0 ; DATA XREF: sub_106C0+CAt0
.data:00012569 SourceString db '\\REGISTRY\\Machine\\System\\CurrentControlSet\\Control\\RestoreList',0 ; DriverEntry+30f0
.data:00012568 db 89h dup(0)
.data:00012569 ; char aVideoBase[]
.data:00012569 aVideoBase db 'VideoBase',0 ; DATA XREF: sub_106C0:loc_10785f0
.data:00012568 db 28h dup(0)
.data:00012569 aWindowsCursors db '<WINDOWS>\\Cursors',0 ; DATA XREF: sub_106C0+20f0
.data:00012568 db 0E8h dup(0)
.data:00012569 aWindowsFonts db '<WINDOWS>\\Fonts',0 ; DATA XREF: sub_106C0+51f0
.data:00012568 db 0E8h dup(0)
.data:00012569 off_12858 dd offset aNtoskrnlExe_2 ; DATA XREF: sub_11042+A1fr
.data:00012568 ; "ntoskrnl.exe"
.data:00012569 aHalDll_1 dd offset aHalDll_1 ; "hal.dll"
.data:00012568 ; "disk.sys"
.data:00012569 aDiskSys dd offset aDiskSys
.data:00012568 ; "atapi.sys"
.data:00012569 aAtapiSys dd offset aAtapiSys

```

Fig 18 - recognizable strings

Both strings are used as an arguments to function

```

.text:000106DA      mov     [ebp+var_14], eax
.text:000106DD      mov     [ebp+var_28], eax
.text:000106E0      push    offset aWindowsCursors ; "<WINDOWS>\\Cursors"
.text:000106E5      lea     eax, [ebp+var_8]
.text:000106E8      push    eax
.text:000106E9      lea     ebx, [ebp+var_4]
.text:000106EC      lea     esi, [ebp+var_20]
.text:000106EF      lea     edi, [ebp+P]
.text:000106F2      call    sub_10400
.text:000106F7      test    al, al
.text:000106F9      jz      loc_10785
.text:000106FF      mov     eax, [ebp+var_8]
.text:00010702      mov     ecx, [ebp+var_4]
.text:00010705      add     eax, 0FFFFFFF8h
.text:00010708      cmp     [ecx+4], eax
.text:0001070B      jbe     loc_107A6
.text:00010711      push    offset aWindowsFonts ; "<WINDOWS>\\Fonts"
.text:00010716      lea     eax, [ebp+var_1C]
.text:00010719      push    eax
.text:0001071A      lea     ebx, [ebp+var_2C]
.text:0001071D      lea     esi, [ebp+var_24]

```

Fig 19 - arguments

Several templates like temp, windows

```

.text:00010E50      ; CODE XREF: sub_10D72+981j
.text:00010E50      loc_10E50: ; sub_10D72+811j
.text:00010E50      cmp     edi, 4
.text:00010E5D      jnz     short loc_10E78
.text:00010E60      push    edi ; MaxCount
.text:00010E62      lea     eax, [esi+1]
.text:00010E63      push    offset aTemp ; "temp"
.text:00010E66      push    eax ; Str1
.text:00010E6B      call    _strnicmp
.text:00010E6C      add     esp, 0Ch
.text:00010E71      test    eax, eax
.text:00010E74      jz      short loc_10EBC
.text:00010E78      loc_10E78: ; CODE XREF: sub_10D72+EE1j
.text:00010E78      cmp     edi, 7
.text:00010E7B      jnz     short loc_10EB7
.text:00010E7D      push    edi ; MaxCount
.text:00010E7E      lea     eax, [esi+1]
.text:00010E81      push    offset aWindows ; "windows"
.text:00010E86      push    eax ; Str1
.text:00010E87      call    _strnicmp
.text:00010E8C      add     esp, 0Ch
.text:00010E8F      test    eax, eax

```

Fig 20 - templates(temp,windows)

This routine dynamically resolves kernel system call addresses by locating the base of `ntoskrnl.exe` and inspecting the System Service Descriptor Table. Execution is guarded to occur only once and includes explicit operating system version checks using `NtBuildNumber`, allowing the malware to adapt to different kernel layouts. By resolving system calls such as extended attribute handlers at runtime rather than importing them statically, the loader avoids detection and maintains compatibility across Windows versions. This behavior is characteristic of Regin's highly defensive kernel-level architecture, where low-level system interaction is carefully abstracted and controlled.

```

.text:00011F48      push    esi
.text:00011F49      push    offset aNtoskrnlExe_1 ; "ntoskrnl.exe"
.text:00011F4E      mov     byte_1288E, 1
.text:00011F55      call    sub_11E8E
.text:00011F5A      mov     esi, eax
.text:00011F5C      test    esi, esi
.text:00011F5E      jz      short loc_11FAF
.text:00011F60      mov     eax, ds:NtBuildNumber
.text:00011F65      cmp     word ptr [eax], 565h
.text:00011F6A      jnz     short loc_11F7B
.text:00011F6C      mov     eax, ds:KServiceDescriptorTable
.text:00011F71      mov     eax, [eax]
.text:00011F73      mov     eax, [eax+270h]
.text:00011F79      jmp     short loc_11F86
.text:00011F7B      ; CODE XREF: sub_11F36+341j
.text:00011F7B      loc_11F7B:
.text:00011F7B      push    offset aNtseteafile ; "NtSetEaFile"
.text:00011F7D      push    esi
.text:00011F81      call    sub_11CBC
.text:00011F86      ; CODE XREF: sub_11F36+431j
.text:00011F86      loc_11F86:
.text:00011F86      push    offset aNtqueryeafile ; "NtQueryEaFile"
.text:00011F88      push    esi
.text:00011F8B      mov     dword_12894, eax
.text:00011F8C      call    sub_11CBC
.text:00011F91      cmp     dword_12894, 0
.text:00011F96      mov     dword_12890, eax
.text:00011F9D      jz      short loc_11FAF
.text:00011FA2      test    eax, eax
.text:00011FA4      jz      short loc_11FAF
.text:00011FA8      mov     byte_1288D, 1

```

Fig 21 - Function call

